

Retrieval-Augmented Generation (RAG) with SPARK

Answer questions about a PDF by retrieving only the relevant chunks - and see why RAG uses a fraction of the tokens (slides 5-6).

Activity is designed by Su Nguyen; Detailed guide is drafted by Claude Cowork.

Before you start

Open SPARK and set an API key in Settings (gear, top-right). The activity uses gemini-2.5-flash; if Gemini is overloaded (a "high demand" error), switch the LLM node's AI Provider to a GPT model and re-run (the temporary endpoint and API key will can be found in the announcement). The PDF used here (automated-formulation.pdf) is in Week 6 > data > paper.

Tool	SPARK - Studio for Prototyping Agent Reasoning & Knowledge
Example	Document Q&A with RAG
Document	automated-formulation.pdf (a research paper)
Idea	Embed the document into a vector DB, retrieve only the top-k relevant chunks, then answer
Why it matters	RAG answered using significantly fewer tokens than feeding the whole file

Step 1 - Understand the RAG pipeline

RAG (Retrieval-Augmented Generation) lets a model answer questions about a document without reading the whole thing every time. The document is split up and converted into embeddings (numeric vectors) stored in a small database; at question time only the most relevant pieces are retrieved and passed to the LLM.

On the Gallery, open the Document Q&A with RAG example. The canvas has two halves:

- Indexing (top): Load Document -> Embed Document -> Create DB. The PDF is embedded and stored in a vector database (docs_db).
- Querying (bottom): My Question -> Embed Question -> Find Context (docs_db, k=3) -> Generate Answer -> Show Answer. The question is embedded, the 3 closest chunks are retrieved, and only those go to the LLM.

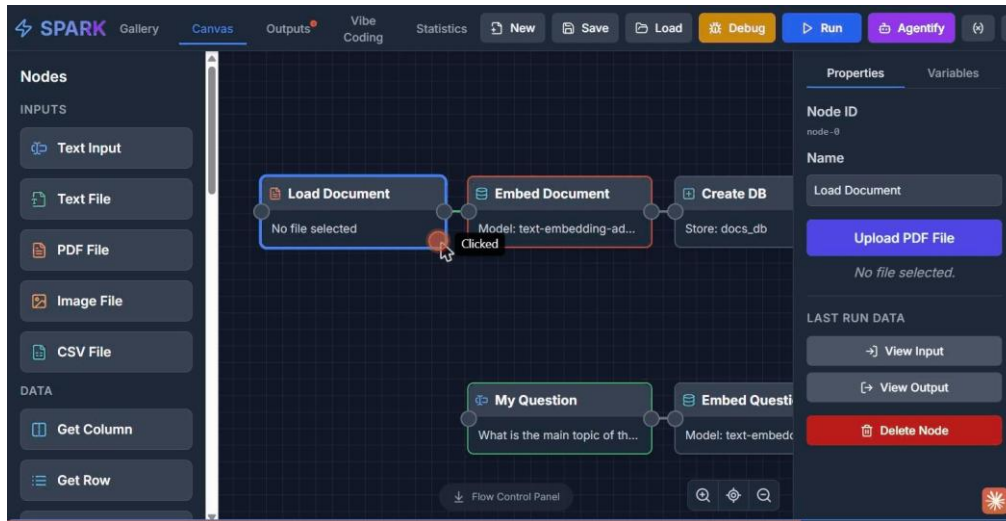


Figure 1 - The RAG pipeline: the document is embedded into a vector DB (top), and the question retrieves the top-3 relevant chunks before the LLM answers (bottom).

Step 2 - Load the PDF and run

1. Click the Load Document node, then Upload PDF File and choose automated-formulation.pdf.
2. Leave the My Question node as "What is the main topic of the document?".
3. Click Run. SPARK embeds the document, builds the vector DB, retrieves the top-3 chunks for the question, and the LLM answers from just those chunks.

The answer is correct but fairly general - the model only saw the retrieved chunks, not the whole paper: it reports the document is about optimization modelling (choosing decisions to minimise or maximise an objective subject to constraints).

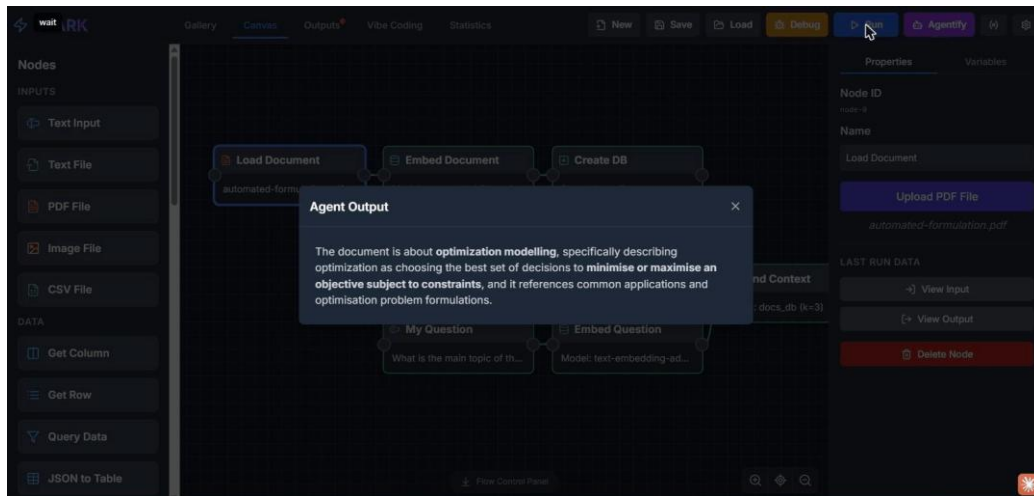


Figure 2 - The RAG answer, generated from only the retrieved context rather than the full document.

Step 3 - Compare: feed the whole file to the LLM

To see what RAG saves, run the alternative approach: send the entire document to the LLM. Open the PDF Q&A example, upload the same automated-formulation.pdf, ask the same question, and Run.

With the full text available, the answer is noticeably richer and more specific - it names the actual contribution: automated prompt optimization for mathematical problem formulations using Small Language Models, introducing FIPO (Feedback-Integrated Prompt Optimiser).

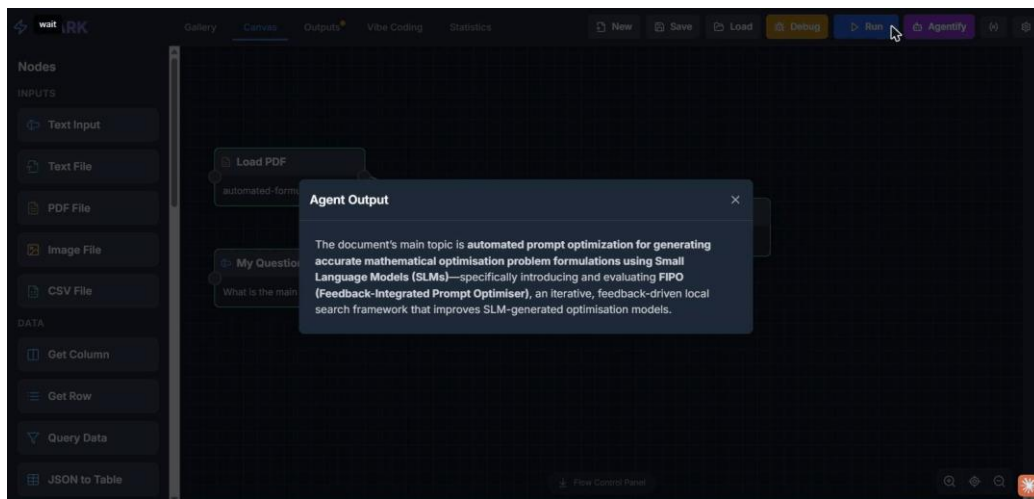


Figure 3 - Feeding the whole file gives a more detailed, specific answer (it names FIPO and SLMs) - but at a cost, as the next step shows.

Step 4 - Read the token consumption

Open the Outputs tab to compare the two runs side by side. Each output shows its input (In) and output (Out) token counts.

Whole file to LLM (Output #5)	In: 17,632 tokens
RAG (Output #4)	In: 407 tokens

RAG used roughly 1/40th of the input tokens. On a long document, or across many questions, that is a large saving in cost and latency. The trade-off is completeness: RAG only sees the retrieved chunks, so for questions that need the whole document the full-file answer can be richer. Choose the approach that fits the task.

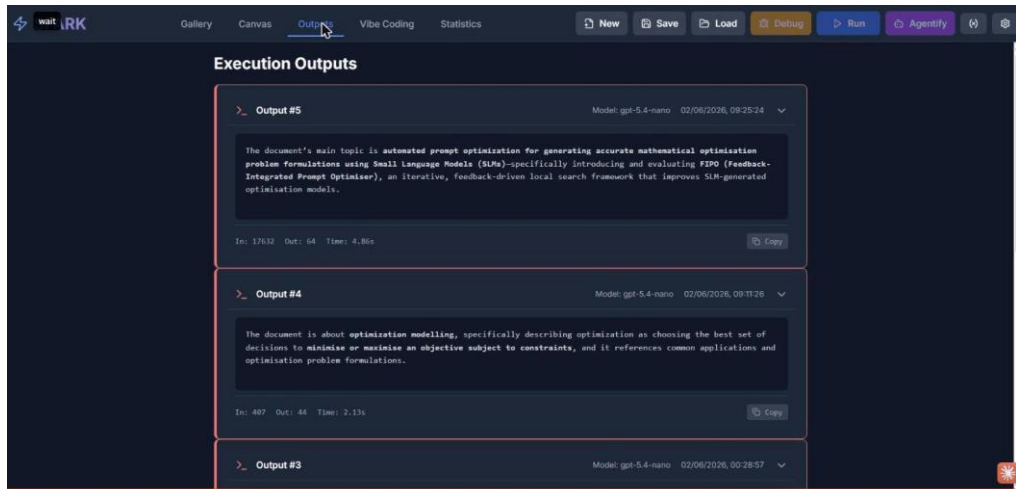


Figure 4 - The Outputs tab: whole-file (17,632 input tokens) vs RAG (407 input tokens) for the same question.

Good to know

- RAG = embed the document once, then retrieve only the relevant chunks per question - cheaper and faster than re-sending the whole file.
- The k value in Find Context (here k=3) sets how many chunks are retrieved; higher k gives more context but more tokens.
- Embedding can be flaky in the current SPARK version (API changes); re-run or switch provider if a node errors.
- RAG trades completeness for efficiency - if an answer seems thin, the needed detail may be in chunks that weren't retrieved.
- Check token counts on the Outputs (or Statistics) tab to make the cost difference concrete for your own documents.